



Dhirubhai Ambani **University** Technology

Formerly DA-IICT

IE-420: Project Report

BIOSTREAM

**Smart Health Monitoring System using STM32 RTOS, Cloud
Dashboard and Mobile Interface**

Group No.: 17

MALAY SIDAPARA - 202201488

SUJAL MAVDIYA - 202201040

KEVIN SHINGALA - 202201465

1. Abstract

Continuous monitoring of vital physiological parameters is crucial for early diagnosis, remote health supervision and emergency response. This project presents **BioStream**, a smart health monitoring system that combines **STM32F411RE** microcontroller, **FreeRTOS**, biomedical sensors and **cloud dashboards** to provide real-time and remote tracking of patient vitals.

The STM32 board acquires Heart Rate, SpO₂, body temperature and body motion using MAX30102, LM35 and MPU6050 sensors. Sensor fusion and formatting are performed under FreeRTOS tasks, and the processed data are streamed via UART to a Python-based **Flask server**. The server pushes live data to a **Firestore Realtime Database** and simultaneously broadcasts updates to a local web dashboard using Socket.IO. Cross-platform **desktop** and **mobile** applications built with Tkinter fetch and display live vitals from Firestore, offering a BioStream-branded interface that clearly visualizes current readings and device status. Additionally, a separate Python analytics module generates a 24-hour health report with statistical metrics and professional-style plots.

The system demonstrates a low-cost, modular and extensible platform for smart healthcare, integrating embedded systems, RTOS scheduling, cloud IoT and user-friendly dashboards.

2. Introduction

2.1 Motivation

Traditional hospital monitoring systems are expensive, non-portable and often confined to intensive care units. For chronic patients, elderly people living alone and post-operative cases, continuous monitoring outside the hospital is highly valuable. Wearable or bedside embedded systems can collect data, but without **real-time connectivity and intuitive dashboards**, the data has limited use.

Recent advances in microcontrollers, low-power sensors and cloud services enable affordable, always-connected health monitoring systems. The aim of BioStream is to leverage these technologies to design a system that:

- Works in **real time** using an RTOS.
- Streams data securely to the **cloud**.
- Provides rich **desktop and mobile dashboards**.

- Supports future integration of **TinyML/Edge AI** for anomaly detection.

2.2 Problem Statement

Design and implement a **Smart Health Monitoring System** that:

1. Collects heart rate, oxygen saturation, body temperature and motion in real time.
 2. Uses an STM32 microcontroller running **FreeRTOS** to manage sensing, formatting and communication tasks.
 3. Streams live data to a **cloud backend (Firebase)** via a Python gateway.
 4. Visualizes vitals on **TFT display (local)** and **desktop/mobile dashboards (remote)**.
 5. Stores historical data for offline analysis and anomaly detection.
-

3. Objectives and Scope

3.1 Main Objectives

- **Real-time sensing:** Use STM32F411RE with FreeRTOS to periodically sample biomedical sensors with deterministic timing.
- **Reliable communication:** Stream formatted vitals to a host PC through UART at 115200 baud and forward them to cloud.
- **Remote visualization:** Implement a BioStream UI on both desktop and mobile that reads from Firebase and displays live values.
- **Data analytics:** Generate offline health reports with statistics and graphs over 24 hours or more of data.

3.2 Scope

- Fully working **end-to-end pipeline** from sensors to cloud dashboards.
 - STM32 is the **primary microcontroller**; ESP32 is used only during early testing and is mentioned as a secondary experimental platform.
 - Edge AI/TinyML is kept as future work; current system focuses on reliable data collection and dashboards.
-

4. System Requirements

4.1 Functional Requirements

1. Read heart rate and SpO₂ using MAX30102.
2. Read temperature using LM35 sensor.
3. Read body acceleration using MPU6050 (X, Y, Z).
4. Package readings into comma-separated string and send via UART every 1 second.
5. Flask server decodes strings, prints diagnostic logs, updates Firebase's `live_vitals` node and emits Socket.IO events for the web dashboard.
6. Tkinter desktop and mobile clients fetch `live_vitals` JSON from Firebase REST endpoint and update GUI values at ~1 Hz.
7. Data analytics script uses either real Firebase history or synthetic demo data to create a 24-hour report with anomaly detection (fever, tachycardia, hypoxia).

4.2 Non-Functional Requirements

- **Real-time behaviour:** Sensor sampling jitter should be minimal; FreeRTOS tasks ensure deterministic execution.
- **Reliability:** System must handle serial disconnections and Firebase errors gracefully and attempt reconnection.
- **Usability:** Dashboards must be easily readable with clear color coding (magenta for HR, cyan for SpO₂, yellow for temperature, neon green for motion).

- **Scalability:** Cloud database design should support multiple patients or devices in future.
 - **Security:** Firebase access is protected by a service account; sensitive key material is kept outside the public report.
-

5. Hardware Design

5.1 Component List:

Sl. No.	Component	Description
1	STM32F411RE Nucleo Board	Main MCU (Cortex-M4, 100 MHz)
2	MAX30102 Pulse Oximeter Sensor	Measures HR and SpO ₂ via I ² C
3	LM35 Temperature Sensor	Analog sensor, 10 mV/°C
4	MPU6050 IMU	3-axis accelerometer + gyroscope via I ² C
5	TFT Display (GC9A01 / similar)	SPI/I ² C circular display for local visualization
6	3.3 V Regulator, resistors, capacitors	Power conditioning and pull-ups
7	RGB LEDs	Status indication

8	USB-UART Connection	STM32 to PC for UART data streaming
---	---------------------	-------------------------------------

5.2 STM32 Pin Mapping

Using the STM32CubeMX pin-out, typical mapping is:

Function	STM32 Pin	Description
I ² C1_SCL	PB8	Shared by MAX30102 + MPU6050
I ² C1_SDA	PB9	Shared bus data
ADC1_IN0 (LM35)	PA0	Analog input
SPI1_SCK	PA5	TFT clock
SPI1_MOSI	PA7	TFT data
SPI1_NSS	PA4	TFT chip select
UART2_TX	PA2	STM32 → PC
UART2_RX	PA3	PC → STM32 (optional)
LED Pins	PA1, PA4, PA9, PC7, PB6	Status and debugging outputs

		main
--	--	------

These pins are configured in MX_ADC1_Init, MX_I2C1_Init, MX_I2C2_Init, MX_SPI1_Init and MX_USART2_UART_Init

5.3 Circuit Description

1. **Power Section:** 5 V from USB is regulated to 3.3 V using AMS1117 or onboard regulators. Decoupling capacitors (0.1 μ F, 10 μ F) are placed near the MCU and sensors.
2. **I²C Bus:** MAX30102 and MPU6050 share SCL and SDA lines pulled up with 4.7 k Ω resistors to 3.3 V.
3. **LM35:** Output connected to PA0; sensor powered from 5 V or 3.3 V, with analogue reference scaled accordingly.
4. **TFT:** Connected via SPI; chip select and control pins assigned to GPIO.
5. **UART-USB:** Nucleo's ST-LINK V2-1 provides virtual COM port, which is used by app.py.

6. System Architecture

6.1 High-Level Architecture

Data Path:

Sensors → STM32 RTOS → UART → Flask + Socket.IO → Firebase → Desktop/Mobile Dashboard

This corresponds to the original assignment flow but adapted to use STM32 as the main MCU and PC as a gateway instead of Wi-Fi directly from MCU.

6.2 Block Diagram

1. **Sensing Layer**
 - MAX30102, LM35, MPU6050 connected to STM32.

2. RTOS Layer (STM32)

- Periodic tasks for sensor reading, formatting and UART transmission.

3. Communication Layer

- UART at 115200 baud to PC.

4. Gateway & Cloud Layer

- Python **app.py** reads serial, pushes to Firebase and broadcasts through Socket.IO.

5. Presentation Layer

- **Web dashboard** (index.html served by Flask).
 - **Desktop app** and **BioStream Mobile app** built using Tkinter and REST API.
-

7. Detailed Software Implementation

7.1 STM32 Firmware (FreeRTOS)

7.1.1 Initialization Phase

main() performs the standard CubeMX initialization:

- **HAL_Init()** – resets peripherals, configures SysTick.
- **SystemClock_Config()** – sets up PLL (HSI → 84 MHz core).
- Peripheral configuration:
 - **MX_GPIO_Init()** – configures all GPIO pins, LEDs, and button.
 - **MX_ADC1_Init()** – configures ADC1 (12-bit resolution, single-conversion mode on channel 0 for LM35).

- `MX_I2C1_Init()` and `MX_I2C2_Init()` – configure I²C busses for MAX30102 and MPU6050.
- `MX_SPI1_Init()` – sets TFT display interface (master mode, 8-bit, CPOL=0, CPHA=1, prescaler, etc.).
- `MX_USART2_UART_Init()` – sets UART2 at 115200 baud, 8N1.

7.1.2 FreeRTOS Scheduler

- A default task `StartDefaultTask` is created using `osThreadDef` and `osThreadCreate`.
- In the generated file, it currently calls `osDelay(1)` in a loop; in a refined system this task would either:
 - Periodically read sensors and update variables, or
 - Spawn separate dedicated tasks such as `SensorTask`, `CommTask`, `DisplayTask`.

A typical expansion of RTOS tasks:

1. **SensorTask (High Priority)**

- Every 100 ms:
 - Trigger ADC conversion for LM35.
 - Read I²C data from MAX30102 and MPU6050.
 - Convert raw values into engineering units (BPM, °C, g).

2. **FormatterTask (Medium Priority)**

- Constructs data string:
`sprintf(buf, "%d,%.1f,%d,%.2f,%.2f,%.2f,%s\r\n", hr, temp,spo2, ax, ay, az, statusString);`

3. **UARTTask (Medium Priority)**

- Sends formatted string using `HAL_UART_Transmit` to the PC.

4. **StatusTask (Low Priority)**

- Blinks LEDs based on connection or alert levels.

7.2 Serial Gateway and Cloud Backend (Flask + Firebase)

The heart of the PC-side gateway is `app.py`, which integrates serial communication, Firebase and Flask.

7.2.1 Firebase Initialization

```
cred = credentials.Certificate("serviceAccountKey.json")

firebase_admin.initialize_app(cred, {

    'databaseURL': 'https://biostream-150bf-default-rtdb.firebaseio.com/'

})

ref = db.reference('live_vitals')
```

- A **service account** JSON file is used to authenticate the app as an admin client.
serviceAccountKey
- A reference to the `live_vitals` node is stored for quick updates.

7.2.2 Flask + Socket.IO Setup

```
app = Flask(__name__)

socketio = SocketIO(app, cors_allowed_origins="*",
    async_mode='threading')
```

- Flask provides a web server and serves `index.html` as the web dashboard.

- Socket.IO enables real-time push updates from the server to any connected web clients.

7.2.3 Serial Reader Thread

A background thread is created:

```
thread = threading.Thread(target=read_serial)

thread.daemon = True

thread.start()
```

The read_serial() function:

- Attempts to open COM5 at 115200 baud (changeable).
- Reads incoming lines from STM32 with ser.readline().
- raw string is decoded and split by commas into 7 parts:
[HR, Temp, SpO2, Ax, Ay, Az, Status].

```
if len(parts) >= 7:

    current_data['hr']    = int(parts[0])

    current_data['temp']  = float(parts[1])

    current_data['spo2']  = int(parts[2])

    current_data['ax']    = float(parts[3])

    current_data['ay']    = float(parts[4])

    current_data['az']    = float(parts[5])

    current_data['status'] = parts[6]
```

- Diagnostic print:
"[STM32] HR:... SpO2:... T:... Motion:(ax,ay,az) | status"
- Emits `update_data` event via Socket.IO to update the browser dashboard.
- Writes `current_data` to Firebase using `ref.set(current_data)`.

Robustness:

- Try/except blocks ensure that serial and Firebase errors do not crash the app; on failure it waits and retries.

7.3 Web Dashboard (Flask Template)

The route in `app.py` simply returns `index.html`:

```
@app.route('/')  
  
def index():  
  
    return render_template('index.html')
```

`index.html` (not provided here, but assumed) subscribes to Socket.IO `update_data` messages and updates DOM elements representing HR, SpO₂, temperature and motion.

Typical features:

- Color-coded cards similar to the desktop/mobile apps.
- A live indicator when Socket.IO connection is active.

7.4 Desktop Dashboard (Tkinter)

The **desktop_app.py** file (structure similar to mobile one) creates a PC dashboard that:

- Periodically calls `requests.get(FIREBASE_URL)` where `FIREBASE_URL` is `.../live_vitals.json`.
- Parses JSON to extract `hr`, `spo2`, `temp`, `ax`, `ay`, `status`.
- Updates labels in a Tkinter window with color-coded fonts.

The theme is dark, matching the BioStream brand.

7.5 BioStream Mobile App (Kivy)

The **BioStreamMobileApp** in `mobile_app.py` simulates a smartphone UI on the laptop.

Key characteristics:

- Window size fixed to **360×700**, similar to an Android phone.
- Header bar:
 - Title: “BioStream” in cyan.
 - Indicator: “● Live” in green.
- Four vertically stacked **cards**:

1. Heart Rate Card

- Left color strip: **#FF4081** (pink).
- Displays label `HEART RATE`, big value (e.g., 72) and units **BPM**.

2. Blood Oxygen Card

- Color strip: **#00E5FF** (cyan).
- Shows % units.

3. Temperature Card

- Color strip: **#FFEB3B** (yellow).
- Shows °C; screenshot shows **27.7°C**.

4. Body Motion Card

- Color strip: **#76FF03** (green).
- Shows text: X:0.05 | Y:0.04 etc.

At the bottom, a status bar displays “**Device Status: Live ...**” colored green when the status string contains “Live;” else orange. The app periodically (every 1 second) spawns a thread to request `live_vitals.json` and update the UI.

This UI corresponds exactly to the screenshot you provided.

7.6 Data Analysis & Health Report

The `data_analysis.py` script has two modes: real Firebase data and mock data.

7.6.1 Data Acquisition

- If `USE_REAL_FIREBASE_DATA = True`, it connects to Firebase, reads the history node and constructs a pandas DataFrame from that JSON (requires history to be recorded by `app.py` in future).
- If `USE_REAL_FIREBASE_DATA = False`, a **mock 24-hour dataset** is generated representing a realistic patient day:
 - Heart rate baseline ≈ 70 BPM with:
 - Lower HR during sleep (-10 BPM from midnight to 6 AM).
 - Jogging spike: $+50$ BPM in a 1-hour morning window (tachycardia event).
 - Temperature baseline $\approx 36.6^{\circ}\text{C}$:
 - Evening fever event: $+1.5^{\circ}\text{C}$ for a time window.
 - SpO_2 baseline $\approx 98\%$:

- Occasionally clipped to minimum 90% to simulate dips.

This dataset becomes a realistic demo for analysis and report plots.

7.6.2 Statistical Summary

Using pandas:

- Average Heart Rate
- Maximum Heart Rate
- Average Temperature
- Maximum Temperature
- Minimum SpO₂

Anomalies are detected:

```
if max_hr > 100: → Tachycardia alert  
  
if max_temp > 37.5: → Fever alert  
  
if min_spo2 < 95: → Hypoxia warning
```

Results are printed in a formatted console report:

```
"BIOSTREAM HEALTH REPORT... Observation Period: 1440 minutes...  
Tachycardia alert, Fever alert..."
```

7.6.3 Visualization

With matplotlib:

- Subplot 1: **HeartRate vs Time** with tachycardia threshold line at 100 BPM.
- Subplot 2: **Temperature vs Time** with fever threshold line at 37.5°C.

- Subplot 3: **SpO₂ vs Time** with hypoxia line at 95%.

This section supports the “**Results and Discussion**” chapter with clear graphs.

8. Testing and Validation

8.1 Unit Testing

- **Serial Gateway:** Verified by printing each parsed raw line and converted fields. Lines with incorrect format are silently ignored.
- **Firebase Connectivity:** On successful initialization, app prints “FIREBASE CONNECTED”; on error, `firebase_active` flag prevents further writes.
- **Mobile / Desktop UI:** Values updated correctly when `live_vitals` node is manually changed in Firebase console.

8.2 Integration Testing

1. STM32 sends test patterns over UART.
2. `app.py` decodes and prints them, updates Firebase.
3. BioStream Mobile & Desktop apps show the same numbers within 1 second.
4. Web dashboard (if open) updates via Socket.IO events.

8.3 Performance

- UART throughput is sufficient for 1 Hz data; lines are typically < 80 bytes.
- Flask + Socket.IO plus Firebase writes occur with latency of ~100–300 ms on a typical connection.
- Tkinter UIs refresh at 0.5–1.0 s intervals giving near-real-time feel.

8.4 Reliability and Error Handling

- Serial reconnection on failure using `outer while True with time.sleep(3)`.
 - Network errors during REST calls in mobile/desktop apps are caught and ignored to keep UI responsive.
-

9. Results and Discussion

9.1 Live Dashboard

The BioStream dashboard shows four main cards:

- **HEART RATE:** Displays BPM in magenta.
- **BLOOD OXYGEN:** Displays % in cyan.
- **TEMPERATURE:** Displays °C in yellow; screenshot shows 27.7°C from LM35 when sensor is at room temperature.
- **BODY MOTION:** Displays accelerometer X and Y readings like `X:0.05 | Y:0.04` in neon green, representing movement in g-units.

The top bar shows “BioStream” and a green “● Live” indicator. This gives a polished, app-like feeling even though it runs on a PC.

9.2 Data Analytics Example

Using the generated 24-hour dataset:

- Average HR $\approx$ 70 BPM; maximum observed $\approx$ 130 BPM during simulated exercise.
- Average body temperature $\approx$ 36.6°C; maximum $\approx$ 38.1°C triggering fever alert.
- Minimum SpO₂ sometimes dips below 95%, resulting in hypoxia warnings.

These values are consistent with realistic physiological ranges and illustrate how BioStream could support clinicians with daily summary reports.

10. Conclusion

The BioStream Smart Health Monitoring System successfully combines **embedded RTOS, cloud IoT and multi-platform dashboards** into a single cohesive project:

- The STM32 microcontroller provides deterministic timing and flexible IO for biomedical sensors.
- Python Flask server bridges the embedded device with **Firebase**, demonstrating a common IoT pattern where a constrained device uses a local gateway.
- Desktop and mobile Tkinter apps, using a unified dark-themed BioStream design, show how simple technologies can imitate modern mobile UIs while consuming real cloud data.
- The analytics module demonstrates the potential for higher level insights, enabling anomaly detection over time.

Overall, BioStream achieves its objective of providing a low-cost, extendable, real-time health monitoring solution suitable as a foundation for further research in smart healthcare devices.

11. Future Work

1. Edge AI on STM32 / ESP32:

- Integrate TensorFlow Lite Micro to perform local arrhythmia or fever detection on the MCU itself, reducing cloud dependency.

2. Wireless Connectivity from Device:

- Replace the UART-PC link with Wi-Fi or BLE directly from a board like ESP32-WROOM; current work with ESP32 during testing can be extended into a full wireless version.

3. Battery-Powered Wearable Form Factor:

- Design a custom PCB, add Li-ion battery + charging circuit and make the system compact and wearable.

4. **Better Security and Multi-User Support:**

- Use per-user auth and store data per patient in Firebase with role-based access.

5. **Doctor Web Portal and Alerts:**

- Add SMS/email/WhatsApp alerts when critical thresholds are crossed

12. Video Link

- 1)  RTES_Project_Part_1.mp4
- 2)  RTES_Project_Part_2_Firebase_kivy_app.mp4